

Deploying a Forecasting Model for Glovo

Production and Deployment Considerations

Charley Baker, Grace Hotchkiss & Ronnie Obeng-Osei

September 15, 2026

This document describes the production deployment of an hourly order-demand forecasting model for one Glovo city (Barcelona). Each Sunday at 23:59 a scheduled job retrains on all available history and forecasts the next 168 hours, which Operations uses to size couriers per slot. This document covers the team and project deployment considerations including: ways of working, effort estimates, the end-to-end AWS architecture and data flows, environments, model build and monitoring, end-user consumption, and the success key performance indicators (KPIs).

From notebooks to production-ready systems

This project bridges that gap by treating the forecasting model not as an isolated experiment but as an operational system that must run reliably every week, handle missing data, detect problems automatically, and support business decisions. Moving from a notebook environment to a production system requires discipline across three dimensions: *engineering* (cloud infrastructure, containerisation, scheduling), *operations* (monitoring, alerting, rollback procedures), and *product thinking* (how users consume the forecast, what success looks like). This deployment plan addresses all three.

Team, roles and ways of working

We operate as a three-person team working in week-long iterations against a shared GitHub repository. Roles are primary but not exclusive; all members review each other's pull requests, and the team pairs on the harder problems (validation design, leakage checks, pipeline wiring). The three roles are:

Member		Primary role	Owns
Grace Hotchkiss: Data & Visualization	Analytics lead	EDA, data cleaning, gap-filling, visualization, end-user dashboard design	
Ronnie Obeng-Osei: Modelling & Engineering	Modelling lead	Feature engineering, model development, backtesting, SMAPE/MSE evaluation, AWS pipeline	
Charley Baker: Analytics & KPIs	Product lead	KPI definition, success criteria, monitoring design, documentation	

Ways of working: We follow trunk-based development with short-lived feature branches. Every merge to `main` passes the format checker and rolling backtest in CI. Communication is

asynchronous where possible (documented decisions, code review comments) with weekly 30-minute synchronous check-ins. Our definition of ‘done’ per sprint is where the code is reviewed, runs end-to-end in the cloud, and the output CSV passes `check_output_format`.

Risk mitigation: In production, small changes can break forecasts. A missing data file, a code update, or a feature engineering mistake can degrade accuracy without anyone noticing. Our ways of working address and mitigate these risks through: version control for all code and data pipelines, automated testing that catches regressions before they reach production, and monitoring that alerts on data drift and forecast degradation.

Effort estimates and how they were agreed

Estimates were sized jointly using planning poker (Agile) on a relative story point scale (1 point $\approx$ half a focused day). We re-estimated after the EDA spike, as the proportion of structurally-zero overnight hours changed the modelling scope. The table below lists feature and task estimates:

Feature / task	Estimate	Basis of estimate
EDA and data cleaning spike	2 pts	Known dataset size; one analyst-day to chart and document patterns
Baselines (naïve, profile)	1 pt	Low risk, mostly groupby logic
Two model families and tuning	3 pts	Highest uncertainty; budgeted for iteration on the zero-hour problem
Rolling backtest harness	2 pts	Reusable; care needed to avoid leakage
AWS architecture and S3 I/O	3 pts	Container, IaC, scheduling, permissions
Monitoring and dashboard	2 pts	CloudWatch alarms plus QuickSight wiring
Documentation, video, write-up	2 pts	Fixed-scope deliverables
Total	15 pts	Approximately two weeks for three people

Sprint velocity is 15 points per sprint (approximately two weeks). The workflow was roughly sequential in the first week, where Grace led the EDA and data cleaning work to produce a clean, gap-filled series that the rest of the team could build on. We then diverged in the second week, with Ronnie driving the modelling and AWS pipeline in parallel with Charley’s work on KPI definition, monitoring design, and documentation.

End-to-end architecture

The job is a single containerised Python application (one image; see the `Dockerfile`). The same artefact runs locally and in the cloud and only the input/output URIs change. Compute is event-driven and ephemeral, so we pay only for the few minutes of weekly training rather than standing servers.

- **Ingest:** Operational orders land in the `raw/` prefix of an S3 data lake (batch export; a Kinesis stream is available if near-real-time ingestion is later required).
- **Schedule:** An EventBridge cron rule fires every Sunday at 23:59 and submits an AWS Batch job (or ECS task).

- **Train and predict:** The container reads all history from S3, retraines the champion model, and writes a 168-row CSV to the `predictions/` prefix; model artefacts and run metrics go to `model/`.
- **Serve:** Operations reads the courier-demand curve through a QuickSight dashboard; the downstream rostering service queries predictions via Athena or a thin API.

End-to-end forecasting architecture on AWS

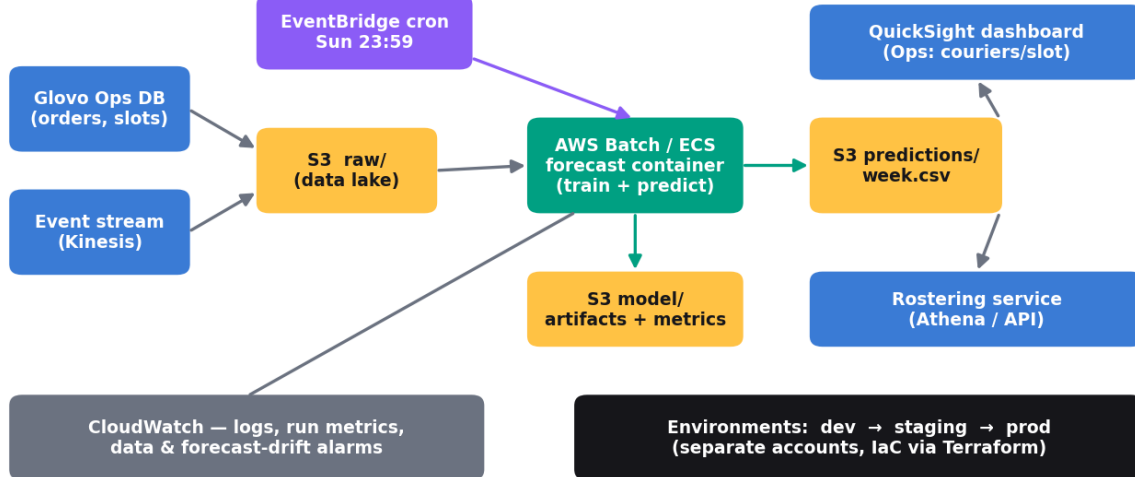


Figure 1: End-to-end forecasting architecture on AWS. Data flows from the Ops database to S3 (raw/prefix), where a scheduled EventBridge rule triggers a containerised Python job weekly. The job reads history, retraines the model, writes predictions to S3 (predictions/prefix), and emits metrics to CloudWatch. Operations users read the forecast via QuickSight and downstream systems query predictions via Athena or a REST API.

Data used and data flows

Source data is the hourly `train_data.csv` for one city (Barcelona), with columns `time`, `orders`, and `city`, covering February 2021 to January 2022 (8,568 hourly slots after gap-filling). Data flows one way: Ops DB → S3 raw → forecast container → S3 predictions → QuickSight and rostering. The container loads from S3 with pandas (via `s3fs`), reindexes to a complete hourly grid, fills 18 missing hours with the day-of-week and hour seasonal mean, engineers features, predicts, and writes back to S3. Predictions are partitioned by forecast date so that QuickSight and Athena always read the latest week.

Environments

Three isolated environments are used, each justified by a distinct need:

- **Development:** engineers iterate against a sampled S3 bucket with manual triggers and verbose logging. Isolation here is important so that experimental code, including half-finished feature branches, never touches production data or schedules. Fast feedback loops are prioritized over reproducibility.

- **Staging:** mirrors production on the full historical dataset. The weekly job runs here first and the output is auto-validated by the format checker and backtest gate before promotion. This environment exists because bugs that only appear at full data scale (memory issues, schema mismatches, timing edge cases) are caught here rather than in production, where a failed Sunday run would leave Operations without a forecast for the week.
- **Production:** the scheduled Sunday job that Operations depends on. Least-privilege IAM, change control, and CloudWatch alarms are enforced here. Promotion requires a manual sign-off in addition to the automated gates, since the cost of a bad forecast (leading to idle couriers or delivery delays) is real and immediate.

All three environments are provisioned from the same Terraform modules to guarantee consistency and eliminate configuration drift between staging and production.

Validation, testing and continuous integration

Preventing bad forecasts is a three-layer problem: the code must be correct, data must be clean, and results must be plausible. Our approach addresses each layer.

Code validation: On every pull request, GitHub Actions runs a format checker (linting, import order), unit tests validating individual components such as feature engineering and the backtest harness, and a rolling backtest that reproduces the Sunday forecast scenario and confirms SMAPE does not regress beyond a tolerance versus the champion. Only when all checks pass does code reach `main`. Promotion from staging to production is further gated: the backtest SMAPE must not degrade more than a pre-agreed threshold (e.g. +1% relative to champion).

Data validation: Before the model runs, a data quality check confirms that the input file arrived and is non-empty, that there are no unexpected nulls or schema changes, and that row counts and slot means are within expected ranges.

Output validation: After prediction, `check_output_format` confirms exactly 168 rows (7 days $\times$ 24 hours) with correct column names and data types and no nulls or negative values.

Building and monitoring the model

Build and CI/CD: The code lives in GitHub. On every pull request, GitHub Actions runs unit tests, the format checker, and the rolling backtest, then builds and pushes the Docker image to ECR. Promotion from staging to production is gated on the backtest SMAPE not regressing beyond an agreed tolerance versus the current champion model.

Monitor: Every weekly run emits structured logs and metrics to CloudWatch:

- **Operational:** job success/failure, runtime, row count of 168, and no-null assertions; a failure pages the platform owner.
- **Forecast quality:** once actuals for a forecast week arrive, a follow-up job computes realised SMAPE and MSE and trends them over time. An alarm fires if SMAPE drifts above an agreed threshold (for example, above 25%).
- **Data drift:** checks on input volume, the share of zero-order hours, and slot means versus the trailing baseline catch upstream data problems before they reach the model.

How end-users consume the solution

The primary users are city Operations planners. Each Monday they open an Amazon QuickSight dashboard showing the predicted orders-per-slot curve for the week, translated into a suggested courier headcount per hour (orders divided by target deliveries-per-courier). They can compare predicted versus realised demand for prior weeks to build trust in the system over time. The rostering product also consumes the predictions automatically, so the forecast feeds both human planners and scheduling algorithms.

Success KPIs and current performance

We separate *model* KPIs (forecast accuracy) from *business* KPIs (operational improvement), some of which are shown in the table below:

KPI	Type	Target
SMAPE on held-out week	Model	< 30% (champion backtest: 25.2%)
MSE on held-out week	Model	Minimised and tracked weekly
Beats seasonal naïve baseline	Model	Benchmark to beat in future iterations
Courier over/under-staffing cost	Business	Reduce versus manual planning
On-time delivery rate	Business	Maintain or improve at lower cost
Weekly job reliability	Ops	> 99% successful Sunday runs
Monitoring alignment	—	Each KPI has a corresponding alarm or dashboard metric

Champion model: Seasonal naïve model with a mean SMAPE of 17.9%, lower than our 20% target. However when looking at MSE, our best performing model is instead the gradient boosting model, which still reports a mean SMAPE of 20.6%, just outside of our target. SARIMA underperforms against the other two models under both metrics. All metrics are computed on open hours only (hours 6–22 inclusive), excluding overnight slots where order volumes are structurally near zero and the metric is undefined. These results establish a clear benchmark; future iterations would look to introduce calendar and lag features to capture the strong intraday and weekly seasonality observed in the EDA.